NinjasNet: una rete siamese per riconoscere se due parole scritte a mano sono state prodotte dallo stesso autore

Marta Cozzi Virginia Ermondi Tommaso Gnone

Abstract

Stabilire se due parole sono state scritte dallo stesso autore rappresenta una sfida rilevante nell'ambito della biometria e dell'analisi della grafia. In questo studio è stata sviluppata NinjasNet, una rete neurale siamese basata su SigNet e modificata per affrontare il compito di verifica dell'autore su parole diverse. Il modello è stato addestrato e testato utilizzando la contrastive loss e la triplet loss. L'adozione della triplet loss ha permesso di ottenere una separazione più efficace tra autori differenti nello spazio degli embedding. Sono stati inoltre analizzati il comportamento della rete in modalità train() ed eval(), l'impatto della data augmentation e l'utilizzo di un classificatore secondario per la classificazione binaria.

1. Introduzione

Stabilire se due parole scritte a mano sono state prodotte dalla stessa persona è un problema rilevante in campi come la biometria e l'analisi della grafia. Per affrontarlo, si è adottato un approccio basato su una rete neurale convoluzionale siamese. Il modello "SigNet" (1) è stato adattato per apprendere le rappresentazioni metriche di immagini di parole. Inizialmente è stata applicata una "contrastive loss", utilizzata per ridurre la distanza tra coppie di immagini dello stesso autore e aumentarla tra coppie di immagini di parole di autori differenti. Successivamente la contrastive loss è stata sostituita con la "triplet loss", che ha migliorato la capacità discriminativa del modello.

Per la contrastive loss, il modello è stato testato con e senza layer di dropout. Con la triplet loss, sono state testate diverse configurazioni per valutare l'impatto di training più lunghi e della data augmentation sulle prestazioni del modello. Le loss sul "training set" e sul "validation set" sono state calcolate sia in modalità "eval()" (2) sia in modalità "train()" (3), per comprendere gli effetti di elementi presenti solo nella modalità di training sulle performance del modello. Per entrambe le loss (contrastive e triplet) sono stati svolti esperimenti anche su un dominio differente,utilizzato come test set, per verificare le capacità di generalizzazione del modello.

Al fine di valutare in modo accurato le prestazioni sul test set, sono stati adottati diversi approcci, inclusi metodi indipendenti dalla scelta della soglia di separazione.

2. Dataset

Per il "training" e la "validation" è stato utilizzato l'IAM Handwriting Database. Il dataset è stato scelto per la sua varietà di stili e per le annotazioni che hanno permesso di generare coppie di parole dello stesso autore o di autori differenti. Per la fase di test è stato impiegato il CVL Database, un set che offre caratteristiche complementari al database IAM. CVL presenta meno autori di IAM (310 e 657, rispettivamente), ma ciascun autore ha contribuito in modo più uniforme. Ogni immagine è stata convertita in scala di grigi, ridimensionata a 155 x 400 pixel, sottoposta a invertimento di colore (da bianco su nero a nero su bianco) e normalizzata con media 0.5 e deviazione standard 0.5. Queste operazioni hanno standardizzato le immagini.

Per la contrastive loss, sono state utilizzate 18.000 coppie di parole per il training set, 4.000 coppie per il validation set e 4.000 coppie per il test set. Per la triplet loss, per il training set sono state utilizzate 12.246 triplette (24.512 dopo la data augmentation), per il validation set 5.247 triplette, per il test set 3.616 triplette. La suddivisione tra training set e validation se è sempre stata eseguita per autore: il 70% degli autori è stato assegnato al training set, il restante 30% al validation set. Una coppia, o tripletta, appartiene a uno dei set solo se tutti gli autori di tutte le parole che la formano appartengono al medesimo set per evitare "leakage" tra i set.

3. Architettura

L'architettura proposta si ispira a una versione di SigNet in cui ogni ramo è costituito da una rete neurale convoluzionale (CNN) ispirata ad AlexNet (4). SigNet, pur mostrando buone prestazioni nel task di verifica delle firme, non è altrettanto efficace nel compito attuale, dove vengono confrontate parole diverse. È stato quindi necessario adattare l'architettura al nuovo dominio. Sono stati introdotti due "residual block" identici consecutivi, ciascuno composto dai seguenti strati:

0. `Conv2d(256, 256, ks=3, st=1, pad=1)`

Le modifiche apportate hanno permesso di ridurre il numero di parametri da 125 milioni a poco meno di 6 milioni. L'architettura finale di NinjasNet può essere riassunta come segue:

- Strato convoluzionale iniziale: 64 filtri 11×11, seguito da `BatchNorm2d`, `ReLU` e `MaxPool2d`, riducendo le dimensioni spaziali.
- Strato convoluzionale intermedio: 128 filtri 5×5 con padding.
- Strato convoluzionale profondo: 256 filtri 3×3, seguito da due residual block identici.
- Strato di pooling adattivo: per ridurre la dimensione spaziale a 1×1.
- Strati fully connected: due strati, riducendo la dimensione prima a 256 e poi a 64, entrambi accompagnati da `ReLU`, `BatchNorm1d` e `Dropout`.
- Normalizzazione L2: applicata all'output finale per ottenere un embedding unitario a 64 dimensioni.

L'intera architettura viene applicata in parallelo a entrambe le immagini di una coppia, producendo due embedding confrontabili. All'inizio dell'addestramento, il learning rate è stato fissato a $lr = 1 \times 10^{-5}$. È stato poi applicato uno "scheduler", lo riducesse automaticamente qualora non migliorasse per due epoche consecutive. Le architetture di SigNet e di NinjasNet sono confrontabili in Figura 1.

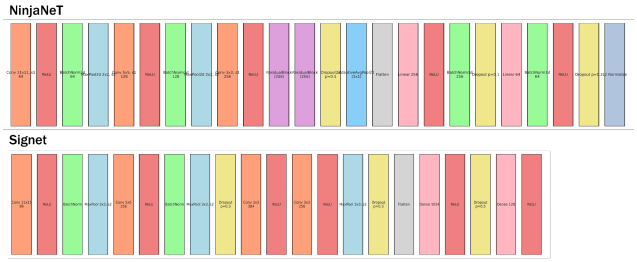


Figura 1. Architettura NinjasNet e SigNet a confronto.

4. Esperimenti e risultati

4.1. Contrastive loss

I dati sono stati organizzati in coppie di immagini, ciascuna etichettata con 1 se le parole erano state scritte dallo stesso autore, 0 altrimenti. La funzione di perdita utilizzata è stata la seguente:

$$\mathcal{L} = (y) \cdot \frac{1}{2} D^2 + (1 - y) \cdot \frac{1}{2} \left[\max(0, m - D)\right]^2 ,$$

dove D rappresenta la distanza euclidea tra gli embedding e il margine è stato fissato a $m = 1.5$, dopo usa serie di prove. Le coppie simili sono penalizzate se D è elevato, poiché la derivata positiva spinge la rete a ridurre tale distanza. Le coppie dissimili sono penalizzate se la loro distanza D è minore del margine m; in questo caso, la derivata è negativa, inducendo la rete ad allontanarle. Se invece sono già distanti almeno quanto il margine, la loss è nulla e la derivata è pari a zero, quindi non viene applicata alcuna penalizzazione.

4.1.1 Effetti del dropout

Durante i test iniziali con il dropout attivo, si è osservato che la loss sul training set risultava costantemente superiore a quella sul validation set (Figura 2), contrariamente a quanto ci si aspetta dalla teoria.

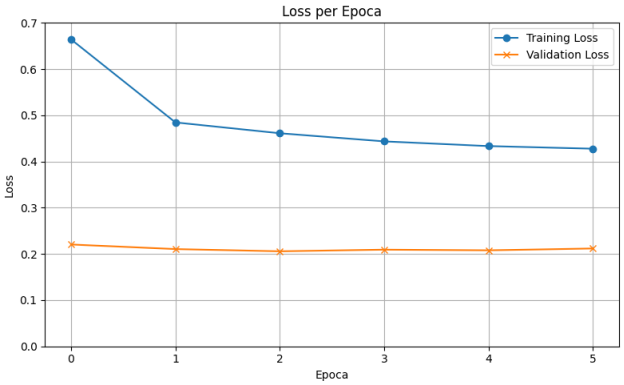


Figura 2. Training loss e validation loss con dropout.

Si è deciso quindi di rimuovere il dropout per valutare il suo effetto sulla loss. Senza dropout e dopo 5 epoche di addestramento, la loss sul training set è diminuita in modo significativo (Figura 3). In teoria, il dropout dovrebbe regolarizzare la rete e ridurre l'overfitting, ma nel caso specifico, prestazioni migliori sono state ottenute eliminandolo. Probabilmente ciò è dovuto al numero limitato di parametri del modello, per cui l'aggiunta di dropout non favorisce la generalizzazione sul validation set, ma rende solo il training più complesso, aumentandone la loss. Inoltre, in presenza di dropout, la validation loss si stabilizza quasi subito dopo la prima epoca, segnale di una capacità di generalizzazione inferiore rispetto al modello senza dropout, che invece continua a migliorare, seppur più lentamente. Prolungando l'addestramento a 10 epoche sulla rete senza dropout, si registra un lieve miglioramento delle performance, con un'accuracy finale sul validation set del 92%. Tuttavia, il persistere del divario tra training e validation loss indica che il dropout non è l'unico elemento a causare un aumento della loss di training.

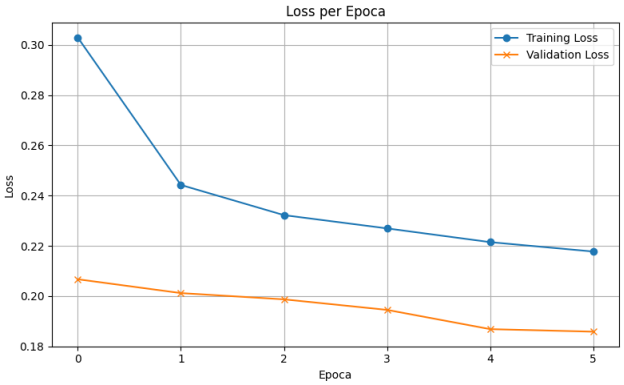


Figura 3. Training loss e validation loss senza dropout.

4.1.2 Effetti della batch normalization

Sia con dropout sia senza, è evidente che la perdita sul training set rimane superiore rispetto a quella sul validation set. Si è ipotizzato che una possibile spiegazione potesse risiedere nella presenza di ulteriori componenti attive solo in modalità train(). È il caso della batch normalization, che, durante il training, calcola media e varianza su ciascun batch, introducendo potenzialmente rumore e instabilità, specialmente con batch piccoli. In fase di validazione, invece, utilizza le medie e varianze globali stimate durante l'addestramento, che sono più stabili:

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i, \quad \sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2$$

$$\mu_{\text{running}} \leftarrow (1-\alpha)\,\mu_{\text{running}} + \alpha\,\mu_B, \quad \sigma^2_{\text{running}} \leftarrow (1-\alpha)\,\sigma^2_{\text{running}} + \alpha\,\sigma_B^2$$

Per analizzare meglio questo comportamento, sono state introdotte:

- `true_train_loss`: perdita calcolata sul training set con il modello in modalità `eval()`.
- `val_train_loss`: perdita calcolata sul validation set con il modello in modalità `train()`.

In Figura 4 è visibile il confronto tra le curve di loss in modalità training ed evaluation per il modello senza dropout.

Quando il modello è in modalità "evaluation", la loss sul training set tende a diminuire per l'assenza di dropout e il diverso comportamento della batch normalization, riducendo il rumore. Per le stesse ragioni, la loss sul validation set può aumentare in modalità di training. Il confronto a coppie delle loss di training e validation, calcolate in modalità train e eval, mostra un andamento coerente con la teoria.

Di seguito si riportano le loss ottenute dopo 6 epoche di addestramento sul modello senza dropout:

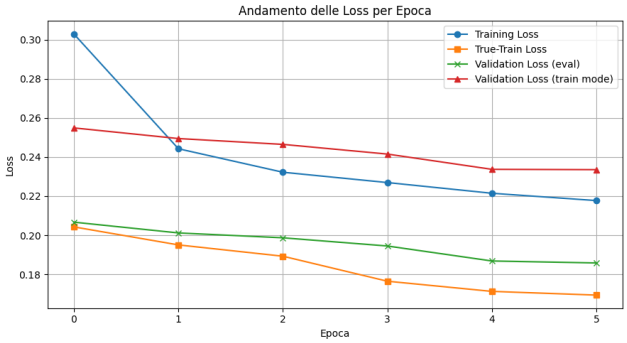


Figura 4. Loss e metriche aggiuntive senza dropout.

- TrainLoss: 0.2177
- TrueTrainLoss: 0.1694
- ValLoss: 0.1858
- ValTrainLoss: 0.2335

4.1.3 Distribuzione degli embedding generati

Per avere un riferimento visivo, è stato prodotto un istogramma che rappresentasse la distribuzione delle distanze tra gli embedding generati per le due classi (simili e dissimili). In Figura 5, è mostrato l'istogramma costruito sul validation set senza dropout. Un comportamento ideale della rete si tradurrebbe in una netta separazione tra le due distribuzioni sul grafico, in cui le coppie simili dovrebbero essere concentrate su valori bassi (a sinistra) e le coppie dissimili su valori alti (a destra). Si nota una parziale sovrapposizione, dovuta principalmente a coppie dissimili che presentano una distanza ridotta. Questo fenomeno suggerisce una certa difficoltà del modello nel discriminare efficacemente tali coppie, compromettendo la capacità di separazione nel caso di esempi negativi.

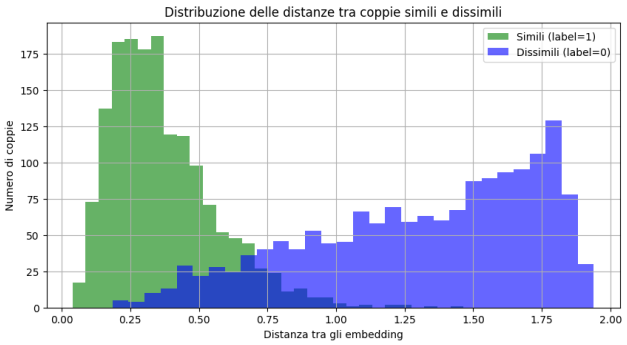


Figura 5. Distribuzione validation set senza dropout.

Si riportano le metriche di classificazione calcolate sul validation set senza dropout (con soglia di decisione $= 0.749573$):

- Accuracy: 0.9128
- Precision: 0.8977
- Recall: 0.9319
- F1-score: 0.9145

L'istogramma è stato calcolato anche sul test set. La distribuzione è riportata in Figura 6.

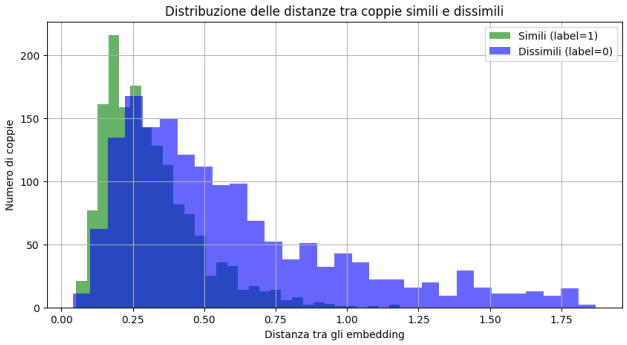


Figura 6. Distribuzione test set senza dropout.

Si nota una significativa diminuzione nella separazione tra le classi, con un'importante sovrapposizione delle distanze. Se le immagini simili vengono rappresentate da distanze coerenti, quelle dissimili non vengono praticamente discriminate.

Mantenendo la soglia fissa calcolata sul validation set (0.749573), si riscontrano i seguenti risultati:

- Accuracy = 0.6200
- Precision = 0.5701
- Recall = 0.9760
- F1-score= 0.7198

Si osserva che la precision è molto bassa, situazione coerente con la distribuzione.

La Figura 7 mostra le distanze tra gli embedding sul validation set introducendo il dropout. Si riscontra una separazione lievemente minore tra le classi rispetto al modello senza dropout.

Metriche di classificazione (soglia di decisione fissata a 0.739451):

- Accuracy = 0.8849
- Precision = 0.8497
- Recall = 0.9352
- F1-score = 0.8904

L'istogramma è stato costruito anche sul test set. La distribuzione è riportata in Figura 8.

Metriche di classificazione (soglia di decisione fissata a 0.739451):

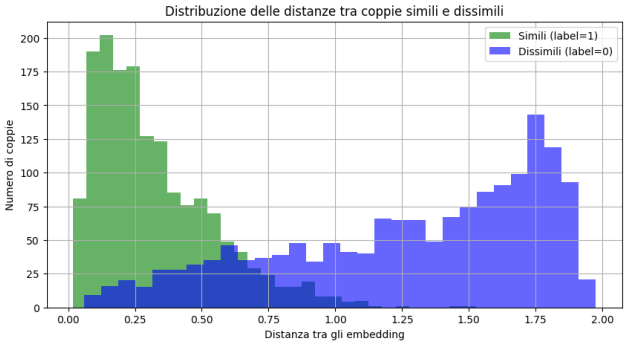


Figura 7. Distribuzione validation set con dropout.

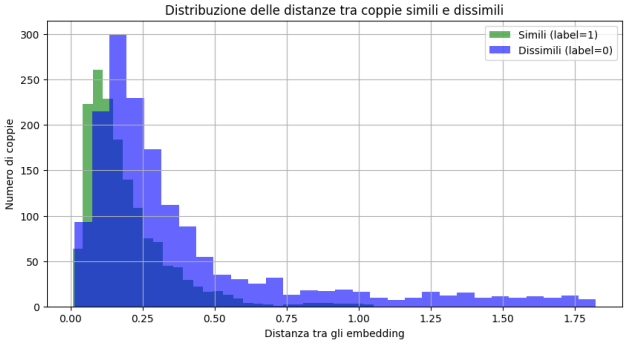


Figura 8. Distribuzione test set con dropout.

- Accuracy = 0.5617
- Precision= 0.5336
- Recall=0.9815
- F1-score = 0.6913

Si nota che la scelta di una soglia ottimale è determinante per ottenere un'accuracy che rappresenti fedelmente la discriminazione tra le due classi anche sul test set.

Sono quindi state calcolate le seguenti metriche, indipendenti dalla soglia fissata:

Senza dropout:

- AUC (Area Under the Curve): 0.7372
- EER (Equal Error Rate): 33.2000% con soglia a 0.351

Con dropout:

- AUC: 0.6906
- EER: 37.1500% con soglia a 0.193

Attraverso l'EER si può ottenere una soglia di separazione ottimale. Confrontando i risultati si riscontrano prestazioni migliori senza livelli di dropout anche sul test set.

4.2. Triplet loss

I dataset di training e validation sono stati riorganizzati in triplette composte da un'immagine "anchor", un'immagine positiva ("positive") appartenente allo stesso autore, e un'immagine negativa ("negative") scritta da un autore differente. La funzione di perdita adottata è stata la seguente:

$$\mathcal{L} = \max\left(0, \text{dist}_{ap} - \text{dist}_{an} + \text{margin}\right),$$

dove dist_{ap} indica la distanza tra l'anchor e il positivo, mentre dist_{an} rappresenta la distanza tra l'anchor e il negativo. Il margine è stato fissato a 1. La rete viene penalizzata quando dist_{ap} non è sufficientemente inferiore a dist_{an}, ovvero quando la differenza tra le due distanze è minore del margine stabilito. Un esempio viene considerato corretto se $\text{dist}_{ap} < \text{dist}_{an}$. L'accuratezza per ciascun batch è calcolata come la porzione di triplette che soddisfa questa condizione. Dopo cinque epoche di addestramento, è stata ottenuta un'accuracy del 94% sul validation set.

4.2.1 Grafici delle loss

È stato analizzato l'andamento della funzione di perdita sul training set e sul validation set per ciascuna epoca di addestramento. Le loss riportate nel grafico in Figura 9 sono state calcolate secondo le modalità precedentemente esposte per la contrastive loss.

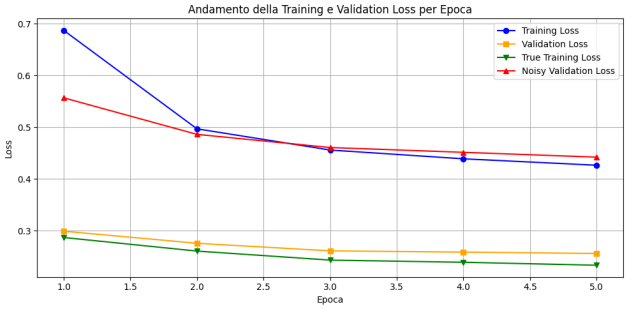


Figura 9. Loss su training e validation set.

4.2.2 Distribuzione degli embedding generati

Sono stati costruiti gli istogrammi delle distanze dist_{ap} e dist_{an}. Per ottenere un'accuracy alta sarebbe sufficiente che la maggior parte delle triplette rispettasse la disuguaglianza $\text{dist}_{ap} < \text{dist}_{an}$, ma idealmente si vorrebbe che la differenza tra le due distanze fosse alta, per garantire una separazione robusta degli esempi positivi da quelli negativi. In Figura 10 è riportata la distribuzione delle distanze sul validation set, in Figura 11 è riportata la distribuzione delle distanze sul training set.

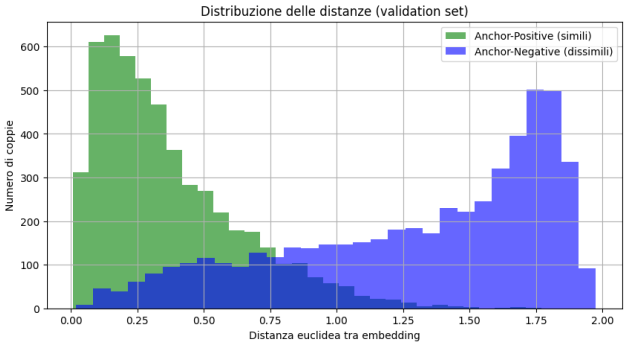


Figura 10. Distribuzione distanze sul validation set.

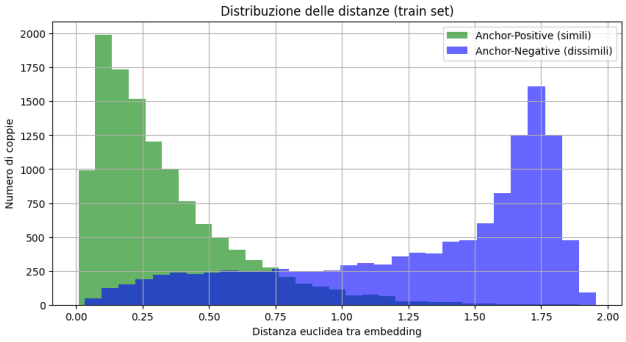


Figura 11. Distribuzione distanze sul training set.

Le separazioni tra dist_{ap} e dist_{an} risultano soddisfacenti. Nonostante non sia possibile operare un confronto diretto tra gli istogrammi della triplet loss e della contrastive loss per via della diversa organizzazione dei set, considerando le coppie anchor-positive e anchor-negative come approssimabili alle coppie simili e dissimili, si osserva una separazione più netta tra le distribuzioni nel caso della triplet loss.

4.2.3 Test su triplette

Nella fase di testing, si è registrato un calo sull'accuracy (80%) e una significativa sovrapposizione tra le distanze anchor-positive (dist_{ap}) e anchor-negative (dist_{an}) sull'istogramma delle distanze, riportato in Figura 12. I risultati hanno suggerito che la rete stesse producendo dei valori di dist_{ap} e dist_{an} tali per cui valeva ancora $\text{dist}_{ap} < \text{dist}_{an}$ per la maggior parte dei casi (accuracy alta), ma con una separazione dei due valori meno netta (sovrapposizione delle distanze).

Anche la separazione sul test set risulta leggermente migliore rispetto a quella ottenuta con la contrastive loss.

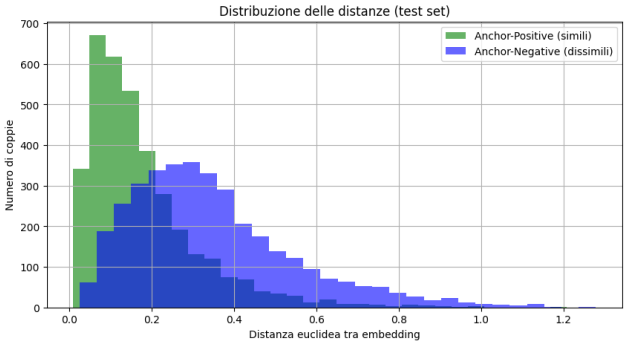


Figura 12. Distribuzione distanze sul test set.

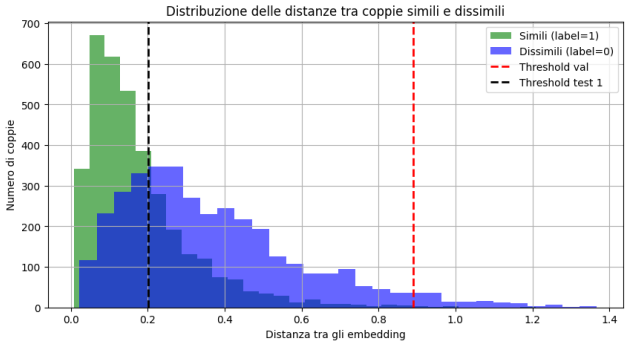


Figura 13. Distribuzione classificazione con soglia.

4.2.4 Test su task binario

L'obiettivo dell'esperimento è stato valutare quanto gli embedding generati dalla rete siamese, addestrata mediante triplet loss, fossero informativi per il compito di classificazione su un dataset di coppie.

Per effettuare la classificazione, è stato necessario definire una soglia decisionale, non necessaria per un training con triplet loss. Una volta calcolata la distanza tra gli embedding delle due immagini nella coppia, se tale distanza superava la soglia prefissata, la coppia veniva classificata come appartenente ad autori diversi; altrimenti la coppia veniva considerata come scritta dallo stesso autore.

La soglia è stata calcolata sul validation set e successivamente applicata al test set per valutarne la generalizzazione (entrambi i set organizzati a coppie).

Il risultato è stato una classificazione fortemente sbilanciata verso la classe "stesso-autore", con elevato numero di falsi positivi. La soglia calcolata sul validation set risultava troppo alta per essere utilizzata sui dati di test. Per il calcolo di una soglia ottimale, il test set è stato diviso a metà, senza condivisione di autori. È stata calcolata una nuova soglia sul test set 1 (prima metà) poi utilizzata sul test set 2 (seconda metà) per le predizioni. I risultati hanno mostrato una soglia molto più piccola rispetto a quella del validation e predizioni migliori e sono rappresentati in Figura 13.

4.2.5 Classificatore binario

Per evitare di fissare una soglia per la classificazione, è stato sperimentato un classificatore caratterizzato da un'architettura semplice, addestrato sul medesimo training set utilizzato per la rete siamese, organizzato in coppie.

La prima versione del classificatore utilizzava la distanza euclidea tra gli embedding generati dalla rete siamese per determinare se una coppia fosse "simile" o "dissimile". Tale approccio ha prodotto risultati di accuratezza molto bassi sul test set, spingendo a considerare ulteriori informazioni per migliorare la capacità discriminativa del classificatore.

Nella versione finale, per ogni coppia di embedding sono state calcolate diverse feature informative: la differenza assoluta tra gli embedding, che cattura la differenza dimensionale; il prodotto elemento per elemento, che esprime la correlazione direzionale; la concatenazione dei due embedding, che permette al classificatore di individuare possibili interazioni; la distanza euclidea, ovvero la norma della differenza; e infine il prodotto scalare, che misura la similarità angolare. Queste feature vengono concatenate in un unico vettore e utilizzate come input del classificatore, che assegna la coppia alla classe "simile" o "dissimile". Questo approccio ha portato a un significativo aumento dell'accuratezza sul test set.

Il test set a coppie è stato passato nel classificatore e sono state calcolate Accuracy (66%) e F1 score (0.74), abbassato dal valore di Precision. Di nuovo la rete ha restituito molti falsi positivi.

4.2.6 Data augmentation

Per migliorare la capacità di generalizzazione della rete, è stata applicata una leggera data augmentation comprendente rotazioni, traslazioni e distorsioni prospettiche, portando il numero di triplette nel training set a 24512.

Il modello è stato riaddestrato utilizzando il training set aumentato per un totale di 9 epoche. L'accuratezza raggiunta sul validation set è stata pari al 93,6%, non evidenziando alcun miglioramento rispetto all'addestramento effettuato senza data augmentation.

Sull'istogramma delle distanze dopo la data augmentation, riportato in Figura 14, si nota un leggero aumento

del numero di esempi anchor-negative con dist_{an} più alti, suggerendo quindi che la rete abbia allontanato ulteriormente gli embedding dei negativi dall'ancora delle triplette.

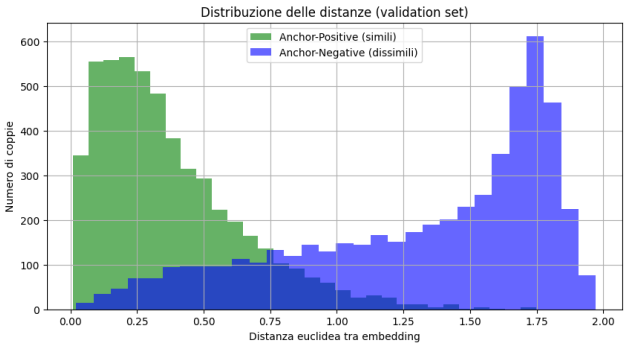


Figura 14. Distribuzione classificazione dopo la data augmentation.

Dopo la data augmentation, la distribuzione delle distanze tra embedding di ancore ed esempi positivi ha anche mostrato un leggero allargamento verso valori più elevati. Questo fenomeno può essere attribuito all'introduzione di una maggiore variabilità intra-classe, che ha reso le immagini positive e le ancore meno simili tra loro, producendo embedding più distanti.

Nonostante questo, l'accuracy invariata suggerisce che la separazione tra classi risulta comunque robusta, mantenendo la disuguaglianza $\text{dist}_{ap} < \text{dist}_{an}$. Inoltre, calcolando la percentuale di violazioni del margine nella triplet loss sul validation set prima e dopo l'augmentation, si osserva una lieve diminuzione nel secondo caso.

4.2.7 Test su triplette

Per quanto riguarda il test sulle triplette, è stato registrato un aumento dell'accuracy di qualche punto percentuale, raggiungendo l'82%. Anche in questo caso, la distribuzione delle dist_{ap} presenta un leggero allargamento verso valori più alti, in accordo con la maggiore variabilità intra-classe introdotta dalla data augmentation.

Analizzando l'istogramma delle distanze sul test set, si riscontra un incremento del numero di coppie con dist_{an} con valori più alti, ma è presente anche un aumento del picco a sinistra, corrispondente a distanze più basse. Tale comportamento, visibile in Figura 15 può essere attribuito sempre alla variabilità introdotta dall'augmentation, che ha avvicinato alcuni esempi negativi alle rispettive ancore.

Al fine di verificare quanto i risultati registrati sul test set fossero legati al dataset aumentato e quanto all'addestramento più lungo, il modello iniziale (train non aumentato) è stato addestrato per ulteriori 4 epoche.

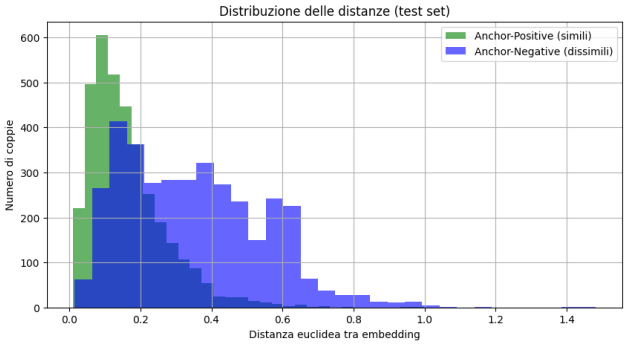


Figura 15. Distribuzione classificazione dopo la data augmentation.

L'accuracy sul validation raggiunta è stata del 94.4%, leggermente più alta del train aumentato, ma senza significativi cambiamenti dal precedente train (non aumentato) di 5 epoche.

L'accuracy sul test set di triplette invece ha registrato un lieve calo arrivando al 79.9% (prima 80.9%). L'istogramma delle distanze sul test set, visibile in Figura 16 ha mostrato anche in questo caso un leggero spostamento verso le distanze più alte per dist_{an}, ma meno marcato rispetto al risultato con il train aumentato.

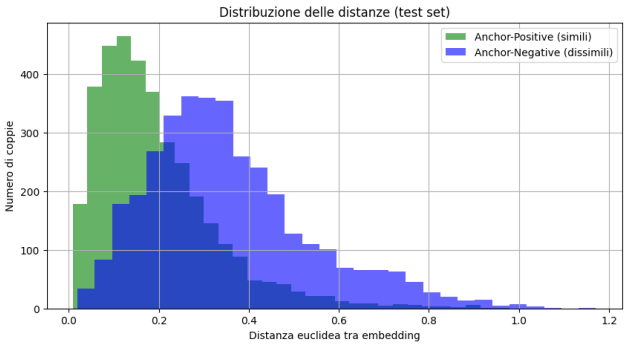


Figura 16. Distribuzione classificazione dopo la data augmentation.

Si deduce quindi che il solo aumento di epoche d'addestramento non porta agli stessi risultati della data augmentation.

In Figura 17 e Figura 18 sono riportati due esempi qualitativi che mostrano diverse triplette. Per le triplette in cui la distanza tra ancora e positivo è maggiore della distanza tra ancora e negativo, è possibile osservare come parole di lunghezza molto diversa possano essere classificate come "dissimili", anche se in realtà apparterrebbero alla classe "simili" (Figura 17).

Per alcune triplette ambigue, si sono verificati alcuni errori (Figura 18).

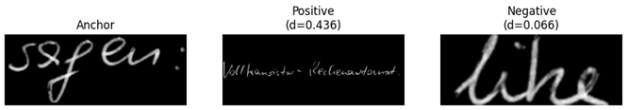


Figura 17. Esempio qualitativo di task su triplette.

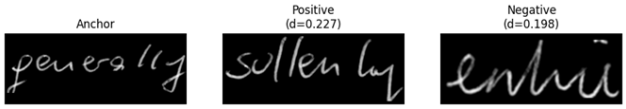


Figura 18. Esempio qualitativo di task su triplette.

4.2.8 Test su task binario con train aumentato

Le predizioni calcolate con soglia fissa sono rimaste pressoché invariate. Le performance sul classificatore, invece, sono leggermente peggiorate. L'introduzione della data augmentation ha aumentato la variabilità nei dati, contribuendo a ridurre l'overfitting sulla triplet loss. Tuttavia, questa maggiore variabilità intraclasse potrebbe aver reso la rete meno stringente nel definire cosa classificare come "simile", peggiorando così le prestazioni nella classificazione binaria. Un istogramma che rappresenta la situazione è riportato in Figura 19

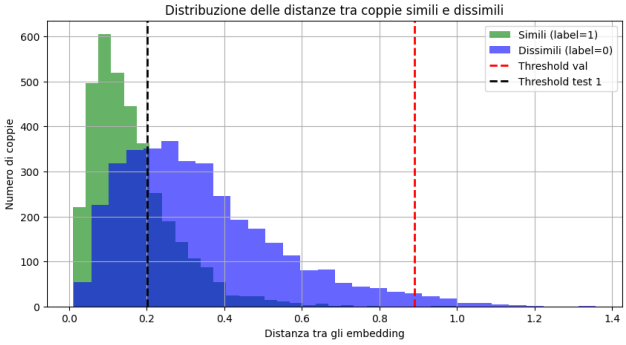


Figura 19. Distribuzione classificazione dopo la data augmentation.

In Figura 20, Figura 21, Figura 22, Figura 23 è possibile osservare alcuni esempi qualitativi della classificazione binaria con soglia sul validation set. In particolare, in Figura 20 è rappresentata una coppia "true positive"; in Figura 21 è rappresentata una coppia "true negative", in cui si nota che le parole presentano zoom diversi tra loro; in Figura 22 è rappresentata una coppia "false positive", in cui il tratto e la dimensione sono in effetti simili; in Figura 23 è rappresentata una coppia "false negative", in cui la dimensione sembra svolgere un ruolo cruciale per la classificazione.



Figura 20. Esempio qualitativo task binario: true positive con soglia su validation set.



Figura 21. Esempio qualitativo task binario: true negative con soglia su validation set.

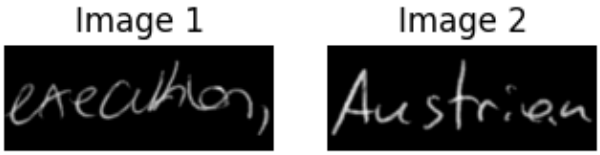


Figura 22. Esempio qualitativo task binario: false positive con soglia su validation set.



Figura 23. Esempio qualitativo task binario: false negative con soglia su validation set.

In Figura 24, Figura 25, Figura 26, Figura 27 è possibile osservare alcuni esempi qualitativi della classificazione binaria con soglia sul test set. In particolare, in Figura 24 è rappresentata una coppia "true positive", in cui la rete è stata capace di effettuare una corretta classificazione nonostante le parole avessero dimensioni diverse; in Figura 25 è rappresentata una coppia "true negative", in cui la rete ha classificato correttamente nonostante le dimensioni simili delle due immagini; in Figura 26 è rappresentata una coppia "false positive", in cui le due parole appaiono effettivamente molto simili tra loro; in Figura 27 è rappresentata una coppia "false negative", in cui la rete non è stata ca-

pace di classificare correttamente, forse a causa delle dimensioni del testo.



Figura 24. Esempio qualitativo task binario: true positive con soglia su test set.



Figura 25. Esempio qualitativo task binario: true negative con soglia su test set.



Figura 26. Esempio qualitativo task binario: false positive con soglia su test set.



Figura 27. Esempio qualitativo task binario: false negative con soglia su test set.

4.2.9 Analisi della difficoltà dei dataset di train e validation

Durante le fasi iniziali degli esperimenti, è stata osservata una loss di validation inferiore rispetto a quella di train, in seguito attribuita alla presenza di dropout e batch normalization. Un'ulteriore verifica è stata eseguita per escludere una possibile differenza di difficoltà tra i due set. Per verificare questa ipotesi, è stata calcolata la percentuale di violazioni del margine della triplet loss sulle triplette di train e validation. Una tripletta è considerata difficile quando la distanza tra esempio positivo e ancora è elevata e/o quella tra esempio negativo e ancora è bassa. Si è quindi ipotizzato che un dataset più complesso presentasse una maggiore percentuale di violazioni della condizione $\text{dist_ap} - \text{dist_an} + 1.0 < 0$.

I risultati hanno evidenziato percentuali di violazione simili su train (44,65%) e validation (48,33%). Per approfondire, è stato generato un boxplot delle differenze $\text{dist_ap} - \text{dist_an}$ per entrambi i set. Un posizionamento del boxplot sopra la soglia di -1 indica violazioni più consistenti. I boxplot relativi a train e validation risultano molto simili in termini di posizione, estensione, mediana e percentuale di outlier e sono visibili in Figura 28.

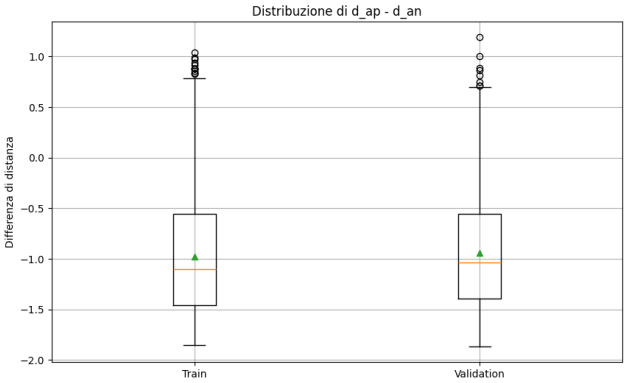


Figura 28. Boxplot su train e validation set.

Infine, la violazione media sulle triplette che infrangono il margine è stata calcolata per entrambi i set, con valori molto vicini (0,53 per train e 0,55 per validation). Si deduce pertanto che non vi siano differenze significative nella difficoltà tra i due dataset. .

5. Conclusioni

In questo lavoro è stata presentata NinjasNet, una rete neurale siamese derivata da SigNet, con l'obiettivo di verificare se due parole manoscritte appartengono allo stesso autore. Gli esperimenti condotti hanno mostrato che l'impiego della triplet loss consente una separazione più efficace nello spazio degli embedding rispetto alla contrastive loss. Nel caso della contrastive loss, l'eliminazione del dropout ha portato a un incremento delle prestazioni, suggerendo che, in questo specifico contesto, la regolarizzazione introdotta da tale tecnica non risulti vantaggiosa.

L'analisi delle funzioni di perdita in modalità train() ed eval() ha evidenziato l'influenza di componenti attive esclusivamente durante l'addestramento, come il dropout e la batch normalization. L'introduzione di una

leggera data augmentation ha determinato una migliore generalizzazione su un dominio differente per il test sulla triplet loss, ma una peggiore classificazione sul task binario.

In generale, il domain shift ha determinato un netto peggioramento delle prestazioni del modello, e la determinazione di una soglia ottimale cross-dominio rappresenta una sfida significativa. Un'adeguata diversificazione dei dati e l'adozione di strategie mirate di domain adaptation potrebbero contribuire a migliorare i risultati. Tuttavia, la limitata disponibilità di dataset specifici per questo task rappresenta un ostacolo rilevante per la generalizzazione del modello.

Note

SigNet Rete neurale convoluzionale siamese progettata per la verifica delle firme. L'architettura si compone di due sottoreti identiche con pesi condivisi, ciascuna costituita da 4 strati convoluzionali ReLU, pooling (max e average); 1 strato fully connected ReLU; 1 strato softmax. Le dimensioni dei filtri convoluzionali variano da 9×9 a 6×16, con un numero crescente di filtri da 32 a 64. L'input è costituito da immagini di firme ridimensionate a 77×190 pixel. Le due sottoreti elaborano ciascuna una delle due immagini di firma in ingresso, e la loro distanza euclidea nel vettore di caratteristiche risultante determina la similarità tra le firme. Link al paper di riferimento: SigNet.

`model.eval()` Modalità in cui si disattiva il dropout e si fissa la batch normalization su statistiche di inferenza.

`model.train()` Modalità che attiva dropout e aggiorna le statistiche di batch normalization.

AlexNet Rete neurale convoluzionale (CNN) profonda composta da 8 strati principali: 5 strati convoluzionali ReLU e operazioni di max-pooling per l'estrazione delle caratteristiche; 3 strati completamente connessi per la classificazione finale; strato di output con funzione softmax per ottenere le probabilità delle classi.